

Shelt Dashboard Documentation

Introduction

The system to be discussed is a full stack automation dashboard designed to fulfil Shelt's requirements by turning all the manual work and cybersecurity services it offers into well designed automation workflows. It is protected by a strong authorization system with roles hierarchy.

The aim of this project is to turn all the manual work done by the SOC team into automated workflows to reduce the time consumption of these tasks and open up space for more productivity and newer services in the workplace.

The SHELDT Dashboard is a full-stack web-based platform designed to support cybersecurity intelligence operations, automated reporting, and investigative workflows. The system integrates multiple security-related tools into a unified interface, enabling users to perform complex tasks such as threat intelligence gathering, domain reconnaissance, report generation, and dark web monitoring.

The platform combines a modern frontend interface with a backend orchestration layer and multiple specialized automation engines. It is designed to streamline operational efficiency by automating repetitive tasks and centralizing data processing and reporting functionalities.

System Objectives

The system is designed to achieve the following objectives:

- Provide a centralized platform for cybersecurity intelligence operations
- Automate the generation of weekly and monthly analytical reports
- Enable reconnaissance activities such as domain analysis and scraping
- Support threat intelligence collection and processing
- Facilitate dark web monitoring and leak detection
- Manage client data and reporting lifecycle
- Ensure secure access control and role-based permissions

System Architecture

The architecture of the SHELDT Dashboard follows a layered structure consisting of a frontend layer, backend layer, data layer, and a set of automation engines. Each layer plays a distinct role in ensuring the system's functionality and scalability.

- **Frontend (Next.js):** handles user interaction and sends API requests
- **Backend (FastAPI):** processes logic, manages jobs, and coordinates modules
- **Database (SQLite):** stores users, reports, logs, and scan results
- **Processing Engines:** execute reporting, scraping, and intelligence tasks

All major workflows follow a consistent pattern:

User Input → API Request → Backend Processing → Background Job → Output Generation
→ Storage → Retrieval

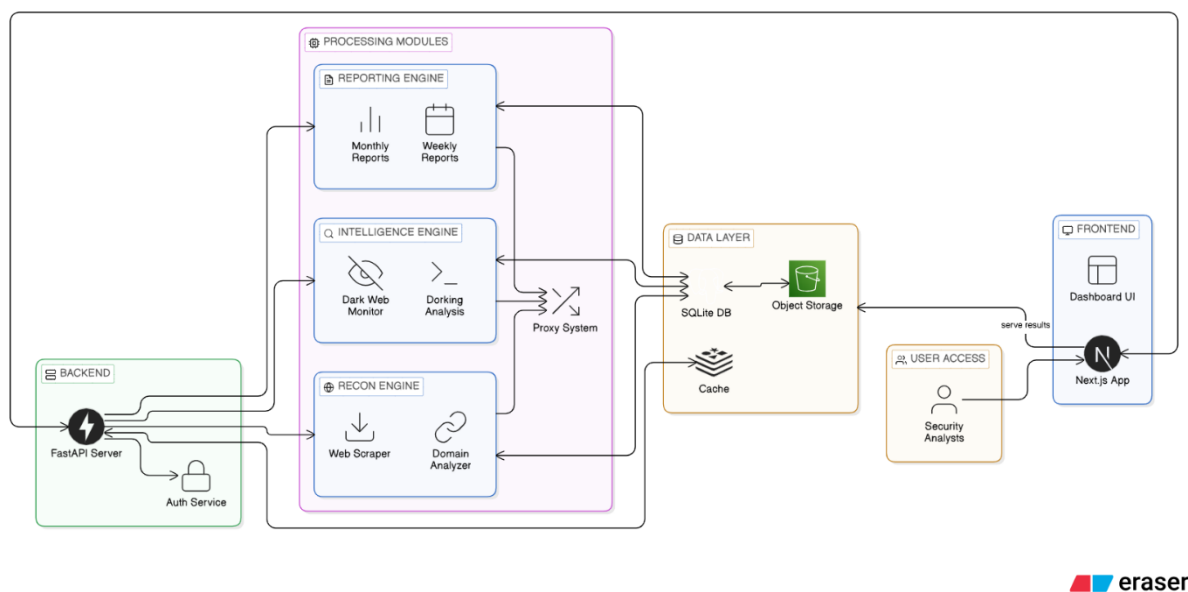


Figure 1: System Architecture

Authentication and Access Control

The system implements a secure authentication mechanism based on bearer tokens. When a user logs in, a token is generated and used to authenticate subsequent requests. Passwords are securely stored using PBKDF2-HMAC-SHA256 hashing with salting, ensuring protection against unauthorized access.

Session management is handled through a combination of in-memory storage and database persistence. While the in-memory cache allows for fast validation, the database ensures that session data is retained across system restarts.

Access control is enforced through a role-based model consisting of two roles: user and manager. Regular users are granted access to operational features such as report generation and analysis tools, while managers have additional privileges, including access to analytics, logs, and user management functionalities.

Background Job Processing

A key aspect of the system is its ability to handle long-running operations asynchronously. Tasks such as report generation and scanning processes are executed as background jobs to prevent blocking user interactions.

Each job follows a lifecycle that includes states such as queued, running, succeeded, or failed. These jobs are managed using in-memory structures and executed in separate threads. Users can monitor the progress of their tasks through dedicated API endpoints that provide real-time status updates.

However, since job data is not persisted in the database, active jobs may be lost if the system restarts, which represents a limitation in the current design.

One of the most important background processing jobs is monthly and weekly reports automation, which due to their heavy weight and content make take up to 10 minutes, and to ensure the user doesn't just sit there waiting for the report to load, I added asynchronous processing, so that they can leave the page and do other things in the dashboard without reports generation progress being lost , in addition to an alert system which sends a notification to the email the user logged in with, which is always their email within the company domain, notifying them that the process is finished and they can download the result , this was done using SMTP.

Core Functional Modules

Weekly Report

The Weekly Report module is responsible for transforming a security scan dataset into a structured operational report enriched with ticketing information. This module plays a central role in bridging raw detection data with real-world remediation tracking.

A key component of this workflow is the integration with REVA, a software used by SHELTER as the primary source of both scan data and ticket information. In this context, a scan refers to a structured Excel dataset generated from a REVA security assessment. This dataset contains detected findings, including severity levels, categories (such as impersonation or domain abuse), affected assets, and status indicators. REVA complements this by providing ticket-level data that represents how these findings are being tracked and resolved within operational workflows.

REVA open (unsolved) tickets are extracted and filtered to the specific client every time a specific script in the project with the REVA API runs, and by the workflow I designed, it runs every time a user generates a weekly report, that way it outputs the newest open

tickets right before the report every time, and this data is included in a sheet in the weekly report excel.

Workflow:

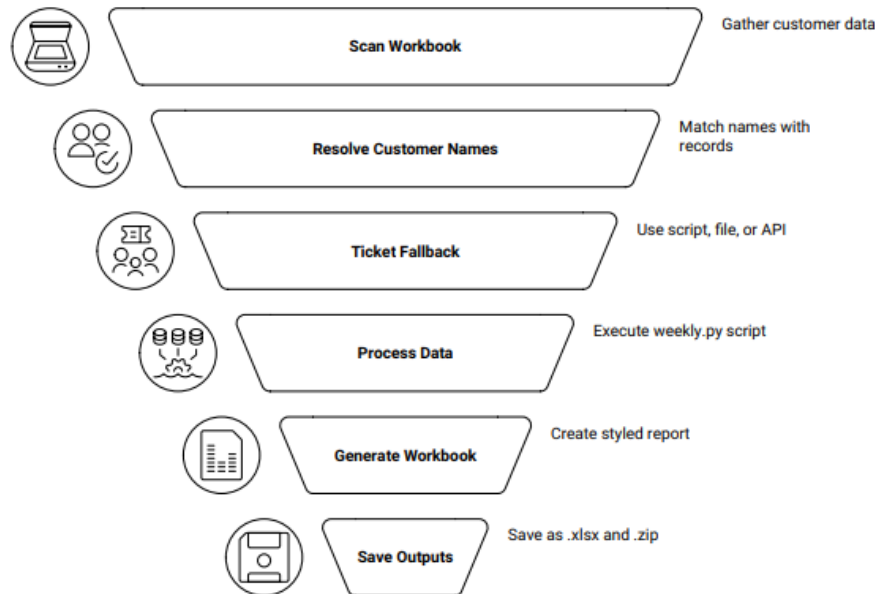


Figure 2: Weekly report generation workflow

1. Scan Workbook + Optional Tickets

- The process begins with scanning the workbook that contains customer data and any optional tickets. This step is crucial for gathering all necessary information for subsequent processing.

2. Customer Name Resolution

- After scanning, the next step is to resolve customer names. This involves matching the names in the workbook with existing records to ensure accuracy and consistency in data handling.

3. Ticket Fallback Chain

- If customer names cannot be resolved directly, the system follows a fallback chain to find the relevant tickets. This chain includes:

4. **Script:** A predefined script that attempts to resolve the ticket based on available data.
5. **Uploaded File:** If the script fails, the system checks any uploaded files for additional information.
6. **REVA API:** As a last resort, the REVA API is called to retrieve ticket information from an external source.

4. **Weekly.py Processing**

- Once the tickets are resolved, the weekly.py script is executed. This script processes the data collected from the previous steps, preparing it for report generation.

5. **Styled Workbook Generation**

- After processing, a styled workbook is generated. This workbook is formatted to enhance readability and presentation, making it suitable for distribution.

6. **Saving Outputs**

- The generated workbook is saved in two formats:
 - **.xlsx**: The standard Excel format for easy access and editing.
 - **.zip**: A compressed file containing the workbook and any related documents, ensuring efficient storage and transfer.

7. **Updating Reports and Logs**

- Finally, the system updates the generated_reports and activity_logs. This step is essential for maintaining a record of all actions taken during the workflow, facilitating future audits and reviews.

This workflow process ensures a systematic approach to ticket management and report generation. By following these steps, organizations can enhance their operational efficiency, maintain accurate records, and provide better service to their customers.

Monthly Report

The Monthly Report module consolidates four required weekly workbooks, together with an optional fifth workbook, into a single branded DOCX report and, a corresponding PDF output. On the frontend, the application accepts weekly report1 through weekly report 4 as mandatory inputs and week 5 as an optional upload. These files are submitted to the asynchronous report-generation endpoint, after which the returned job identifier is stored for status tracking and periodically polled until processing is complete.

On the backend, all uploaded weekly files are written to a temporary run-specific directory before a separate Python subprocess is launched to execute the monthly reporting engine. This subprocess invokes `smart_report_engine/report_builder.py`` with the input Excel directory, the target DOCX output path, and the report template path as its primary arguments. Executing the reporting engine in an isolated process

improves fault containment by preventing report-generation errors from destabilizing the main FastAPI application. Once execution finishes, the backend checks for generated `.docx` and `.pdf` files, copies any successful outputs into the main output directory, and marks the task as failed if no valid report artifact is produced. Successfully generated files are then registered in `generated_reports` under the `monthly` report type, and the system records the activity while optionally issuing an email notification.

The monthly reporting engine operates through four stages, as summarized below.

Stage	Purpose	Core Functionality
1. Workbook Loading	Extract data from the uploaded weekly workbooks	parses workbook content and returns structured findings, metadata, and risk information
2. Canonicalization	Normalize and merge findings across reporting periods	deduplicates findings across weeks and standardizes risk labels
3. Metrics Computation	Produce aggregate statistics and overall posture indicators	calculates severity counts, category breakdowns, trends, and posture classifications
4. Template Population	Assemble the final report document	The engine opens the DOCX template and inserts tables, charts, screenshots, branding, and navigational elements

In the workbook loading stage, the **`load_workbook_data()`** function from **`excel_ingestion.py`** parses each weekly workbook and converts the underlying spreadsheet content into structured records. These records include findings, associated metadata, and risk-related information, thereby establishing the normalized input required for downstream reporting steps.

During canonicalization, **`build_canonical_findings()`** from **`section_mapping.py`** merges findings across all submitted weeks, removes duplicates, and standardizes risk values into a controlled severity set consisting of critical, high, medium, low, safe, and informational. The engine then applies a severity priority mode, ensuring that findings can be consistently sorted and presented throughout the final report.

The metrics computation stage, implemented through **`compute_metrics()`** function in **`metrics_engine.py`**, it transforms canonical findings into quantitative reporting outputs. These outputs include total counts by severity, category level distributions,

trend indicators, and an overall risk posture classification. In report language, the underlying calculations can be expressed through the following formulas.

Metric	Formula
Total Findings	Critical + High + Medium + Low + Safe + Informational
Severity Percentage	$(\text{Findings in a severity level} / \text{Total Findings}) \times 100$
Category Share	$(\text{Findings in a category} / \text{Total Findings}) \times 100$
Weighted Risk Indicator	$(\text{Critical} \times 4 + \text{High} \times 3 + \text{Medium} \times 2 + \text{Low} \times 1) / \text{Total Findings}$

In the template population stage, the engine opens **shelt template.docx** and programmatically populates it with the computed reporting data. This includes the insertion of color coded tables, risk specific sections, visual charts, evidence screenshots ,a dynamic table of contents, and customer branding assets such as the appropriate logo resolved from the customer assets directory.

Since there is no screenshots in weekly report, screenshots are scraped by entering the links of social media extracted from reports and taking screenshots then inserting them in the report under the social media, domain, leak, or impersonation it represents.

Graph generation forms part of the report construction workflow. The engine produces graphs using pandas library to illustrate risk distribution and embeds these visuals directly into the DOCX document.

Logos are stored in a specific directory all titled the name of the client, so whenever weeklys are imported, the name extraction we previously discussed happens and then the name is automatically matched to the photo with the same or very similar title, which makes logo insertion of the client on the first page of the report also automated.

Overall, the Monthly Report module is designed as a structured, asynchronous reporting pipeline that ingests weekly workbook data, consolidates and normalizes findings, computes reporting metrics, and generates a professionally formatted deliverable through automated template population. This architecture supports both operational resilience and consistency of output, while also enabling branded, data-driven reporting at scale.

Dark Web Leaks Module

The Dark Web Leaks module is the most complex intelligence workflow in the platform. It combines surface web querying, public breach correlation, GitHub exposure detection, certificate transparency enrichment, optional Tor based onion search, direct crawling of known onion resources, and impersonation domain analysis.

The frontend page accepts company name, domain, keywords, and mode toggles such as surface-web search, dark-web search, and impersonation detection. It also provides Tor status controls through **GET /darkweb/tor-status** and **POST /darkweb/tor-start**.

When a scan is launched through **POST /darkweb/scan**, the frontend polls the background job and later displays results in separate tabs for leaks, mentions, and impersonations.

The backend first stores the scan configuration in **darkweb_scans**, then starts **_run_darkweb_scan(...)**. The pipeline is divided into four phases.

Phase 1 is surface-web intelligence. **_darkweb_search_paste_sites()** runs multiple sourcespecific searches, including paste-site searches through DuckDuckGo HTML, credential-pattern searches, GitHub code search, crt.sh lookups, and HavelBeenPwned breach correlation. It also performs broader mention searches using leak related search terms. There are six source types in this phase:

. Paste Site Search

The system queries paste-sharing platforms using search engines and parses results to extract potential leaks.

Logic:

- build search query → send request → parse HTML → extract links

2. Credential Pattern Detection

The system searches for structured patterns that indicate leaked credentials.

Patterns detected:

email:password

username:password

Logic:

- retrieve content → apply regex pattern matching → extract matches

3. GitHub Search

Searches public repositories for exposed data.

Logic:

- query GitHub search endpoints
- parse repository results
- extract sensitive strings (keys, configs)

4. Certificate Transparency (crt.sh)

The system queries certificate logs to discover domains.

Logic:

- send request to crt.sh
- parse returned domain list
- filter relevant domains

5. Breach Database Check (HIBP)

Checks if emails/domains exist in known breach datasets.

Logic:

- send API request
- match against breach records
- extract results

6. General Mention Search

Searches for mentions of the company using search engines.

Logic:

- generate keyword-based queries
- collect results
- filter for relevance

Phase 2 is Tor-based onion search when dark web mode is enabled. The backend ensures Tor is running by searching for tor.exe in several known locations, launching it with **--SocksPort 9050**, and waiting for the **SOCKS** port to become available. It then creates a Tor-backed **requests.Session** using **socks5h://127.0.0.1:9050**, which ensures DNS resolution happens through Tor instead of leaking locally.

The system queries onion aware search engines such as Ahmia and Torch, can use a **clearnet DarkSearch** API, and can directly crawl known onion paste sites and forums. The detailed file explains that Ahmia handling includes CSRF-token extraction, redirect decoding, and a fallback to clearnet Ahmia if the onion endpoint is unreachable.

Ahmia is used as a search engine to discover.onion (dark web) resources. Since Ahmia has both clearnet and onion endpoints, the system interacts with it programmatically rather than manually browsing. However, Ahmia is not a simple “GET request” system, it includes security protections like: CSRF tokens, redirects, and request validation, thus, the following step by step logic is implemented:

1. Send initial request to Ahmia search page
2. Receive HTML response
3. Parse HTML
4. Extract CSRF token

5. Store token in session
6. Use token in next search request

Phase 3 is impersonation domain detection when the user enables that mode and supplies a domain. `_darkweb_generate_typo_domains(base_domain)` creates large numbers of variations using deletion, adjacent swap, duplication, homoglyph replacement, dot insertion, TLD swaps, and dash insertion. Each candidate domain is then checked by DNS lookup and HTTP probing. `_darkweb_score_impersonation(...)` computes a 0–100 risk score based on whether the domain is live, whether it resolves to an IP, whether HTTP is successful, whether the page title contains the company name, and whether suspicious keywords such as login, verify, or password appear. The file provides the exact weighting used:

- +40 if live
- +10 if resolved
- +10 if HTTP 200–399
- +20 if company name appears in title
- +15 if the title contains suspicious login words
- +5 for homoglyph variation type

And this makes it capped at 100.

Phase 4 is persistence. Findings are written into four dedicated tables: **darkweb_leaks**, **darkweb_mentions**, **darkweb_impersonations**, and **darkweb_search_queries**.

While **darkweb_scans** is updated to completed status. This database design is one of the strongest parts of the system because it keeps different result types separate and makes export and UI presentation much cleaner. The export endpoint later creates a styled multi sheet Excel workbook and registers it as **report_type='darkweb_scan'**.

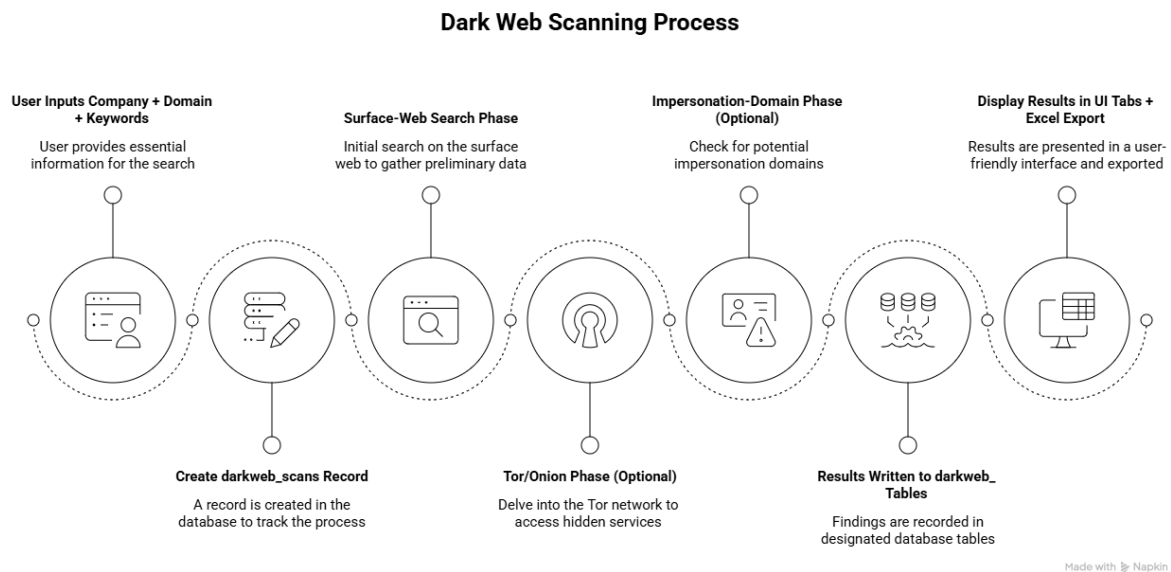


Figure 3: Darkweb Scanning Process

Domain Listing Module

The Domain Listing module generates a threat surface map of domains that may impersonate a target brand. The frontend accepts a target brand and optional official domains, submits them to /domain-listing, and supports export **through /domain-listing/export**. Results can be filtered by risk level and live status.

The backend begins by generating brand variations of eight tiers: base slug, separator variants, homoglyph substitutions, vowel swaps, duplicated letters, character omission, adjacent transposition, and prefix/suffix squatting. These variation slugs are then combined with TLD permutations such as .com, .net, .org, .io, .co, .online, .xyz, .site, .app, .info, .dev, .tech, .club, .live, .store, and region-specific values like .bank, .ng, .co.uk, .com.ng, .co.za, and .com.br.

Next, the module enriches candidates through **crt.sh** by searching for certificates containing the target keyword. After candidate generation and certificate discovery, the system uses a **ThreadPoolExecutor(max_workers=25)** to run DNS resolution and HTTP probing in parallel. DNS resolution uses **socket.getaddrinfo(...)**, while HTTP probing tries HTTPS HEAD, HTTPS GET, HTTP HEAD, and HTTP GET with short timeouts. This parallel design is important because the candidate set can become large very quickly.

Risk classification is then applied with a simple but practical rule set:

- official if the domain is in the official set
- high if it is live and has a certificate
- medium if it is live but no certificate is detected

- low if DNS resolves but HTTP does not
- not registered if neither resolves

The final output is shown interactively and can also be exported into a two sheet Excel workbook containing summary metrics and full domain results.

Social media Scraping Module

This scraping module is an impersonation search system designed to look for fake accounts across Instagram, Facebook, TikTok, Youtube, and Telegram. It uses a browser based scraper, per client keyword configuration, stored session state, username candidate generation, and screenshot based evidence collection.

The frontend allows the user to input a target company or individual, which is then sent as an API request to the backend. The system generates multiple username variations from the input by modifying the spelling and combining it with common keywords such as “official”, “support”, or “team”. These variations are adapted to match platform-specific constraints before being checked.

Some of SHELTY’s clients have specific keywords they would like to search for in addition to the ones already in the script, while other clients don’t, and for that I added a folder that stores excel sheets with keywords for specific clients, so whenever an input is entered, the program walks through these excels one by one to check whether any of their titles matches the one in the input, and extracts the keywords and searches for them accordingly. In addition to that, to make the use of this platform less complicated, I added a section for them to attach new excel sheets for new keywords in case they got new clients, or even if a previous client decided they want to add one now.

On the backend, the scraping process is executed through a subprocess that runs the **hunter_final.py** script. Before execution, the system prepares the environment by normalizing the target input, then loading stored browser session files (such as platform-specific session states), which allow it to operate within an authenticated context without requiring repeated logins. It is important too know that Tiktok, Youtube, and Telegram were easily scraped, and human checks pop up that it encounters sometimes due to repeated requests are simply bypassed by refreshing the page, however Instagram and Facebook couldn’t be scraped until the creation of a fake account for each, logging in once, and saving the session cookies in a file, that way no more login popups would appear.

The scraper generates a large set of username candidates based on the target name and keywords, then visits each candidate profile using browser automation. It validates whether the account exists and whether it is relevant, and captures screenshots as evidence. These screenshots are stabilized before capture and validated to ensure they are usable.

All findings are written into an Excel workbook containing details such as username, profile URL, platform, and associated screenshot. If no valid results are found, the system generates a placeholder workbook to ensure consistent output. The final report is saved in the output directory and registered in the system database for later access.

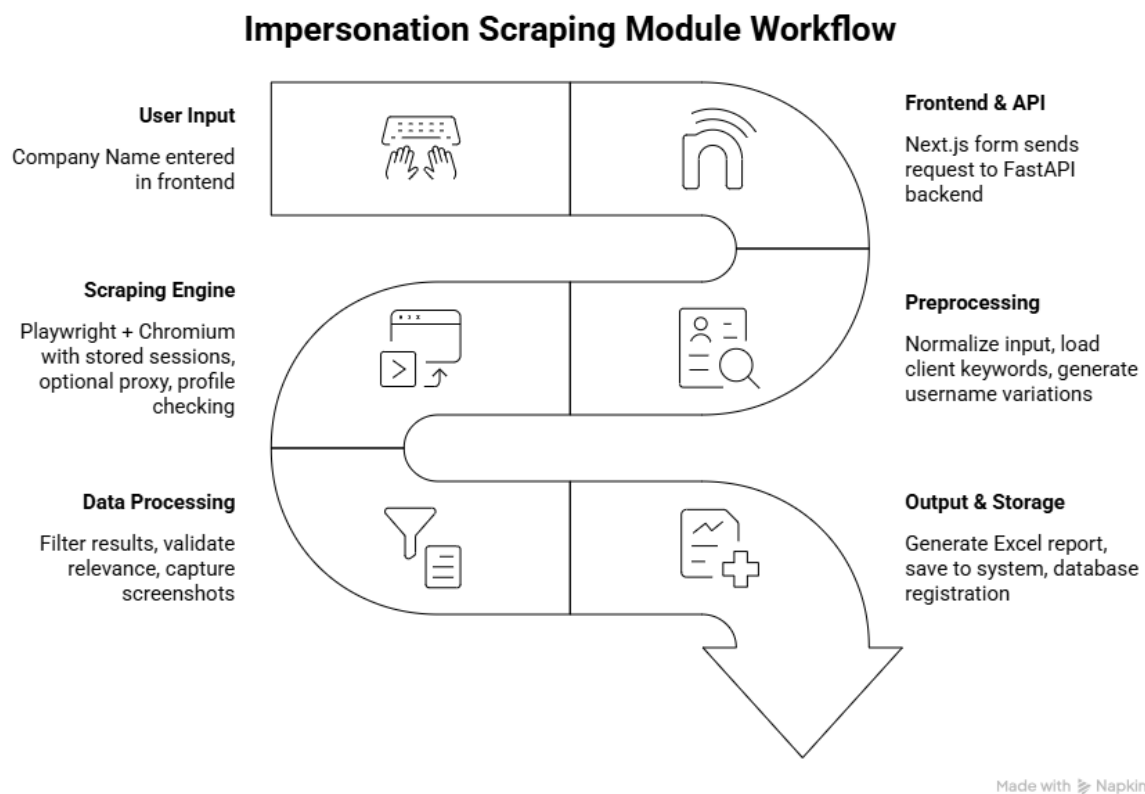


Figure 4: Impersonation Scraping Module Workflow

Google Dorking Module

The Google Dorking module performs structured search-engine reconnaissance. The frontend sends a target brand and optional official domain to **POST /generate-google-dorking** and displays counts such as number of queries, collected links, and screenshots.

The backend builds a dork query set from the target and optional domain, including brand variations, typos, homoglyphs, and category-based query patterns. Query volumes are capped using environment controls such as **DORK_MAX_QUERIES**, **DORK_COLLECT_MAX_QUERIES**, **DORK_COLLECT_RESULTS_PER_QUERY**, **DORK_MAX_TARGET_VARIATIONS**, and **DORK_MAX_SCREENSHOTS**. It then collects evidence from multiple engines: DuckDuckGo HTML, Brave, Mojeek, Google parsing, optional Serper API, and Bing. Results are filtered, screenshots are captured where possible, and the final evidence is exported to an Excel workbook.

The system generates a wide range of search queries (“dorks”) based on the target brand, its variations, and common exposure patterns. These queries are designed to identify publicly accessible sensitive information, misconfigurations, or leaked data.

Some examples of the main dork types used include:

site:example.com filetype:pdf

→ Finds publicly exposed documents on the target domain

"company name" password leak

→ Searches for mentions of leaked credentials

intitle:"index of" "company name"

→ Detects open directories containing company related files

site:example.com inurl:login

→ Identifies login portals that may be exposed

"company name" api key

→ Searches for possible exposed API keys

site:github.com "company name" password

→ Finds potential credential exposure in public repositories

filetype:xlsx OR filetype:csv "company name"

→ Detects exposed data files

These examples represent only a subset of the queries used. In practice, the system generates many variations by combining:

- brand name variations
- typos and homoglyphs
- sensitive keywords (password, confidential, backup)
- file types and URL patterns

This results in a much larger query set, allowing the system to cover a broad range of potential exposure scenarios.

TTI (Threat Intelligence) Module

The threat intelligence module is a simple addition that has a saved excel sheet which includes SHELTY's client and big providers each client takes services from (Microsoft, Oracle, Cisco, etc). The TTI script holds links that provide daily vulnerabilities of these international tech providers, the script then puts the vulnerabilities of the providers each client takes services from and creates an html template to display them, which is also turned into a pdf after. So, every run generates an html and pdf version of the template.

Both html and pdf versions include pressable buttons on each vulnerability that takes users to its resource. The outputs also include a subscribe and unsubscribe buttons that when pressed take the user to the email of the technical manager at SHELT , with either “I want to subscribe” or “I want to unsubscribe”

The UI also has the option to enter a new excel and override the previous one in case clients change their vendors or new clients came in.

Proxy Chain Module

The Proxy Chain module provides routing infrastructure for modules that need controlled outbound network paths, especially scraping and dark web operations. The frontend page supports both auto proxy mode and manual proxy entry. Users can create profiles, import proxies in bulk, validate all entries, test connections, rotate to the next proxy, and export ProxyChains compatible configuration files for Kali Linux usage.

The backend stores configuration in two tables. **proxy_profiles** stores the main profile definition, including type, host, port, credentials, rotation mode, source mode, and status. **proxy_pool** stores the actual proxy endpoints associated with a profile, including IP, port, type, status, and last_used. This split is important because it separates the logical profile from the rotating pool of concrete proxies.

Connection testing is performed by **_test_proxy_connection(...)**, which tries HTTPS HEAD, HTTP HEAD, and HTTPS GET against **httpbin.org/ip**. If the proxy works, the response reveals the exit IP.

Auto fetch mode pulls proxy lists from public GitHub repositories for HTTP, SOCKS4, and SOCKS5 sources, parses each entry, validates structure, and deduplicates proxies using **type:ip:port**. Working proxies are stored back into the pool. Rotation uses a least recently used approach by selecting the oldest active **last_used** value first and then updating that timestamp after selection. If no active pool entry exists, the module falls back to the profile’s own host and port.

The exported ProxyChains configuration uses settings such as **dynamic_chain**, **proxy_dns**, timeout values, and a **[ProxyList]** section containing the profile or pooled endpoints. This makes the module useful not just inside the dashboard, but also for external investigative tooling running on Kali Linux.

When users want a proxy to cover their IP address while working on any of the previously stated modules, they go into this page and start a rotation until a proper one from some open source is found or add their own and they turn it on. They have the choice to turn it off, or change to another one whenever they want.

Previous Reports

The Previous Reports module acts as the archive and retrieval layer for all outputs. The frontend loads grouped artifacts from **/reports**, supports filtering by report type and date, and exposes download buttons. On the backend, **GET /reports** merges two

sources: database records from `generated_reports` and a direct filesystem scan of the output folder. This allows the system to remain resilient even if metadata is incomplete. Downloads are served through `/download/{filename}` with path safety checks.

The UI also offers filtering by report type, and generation date time frames.

Client Info

The Client Info module manages the client registry, lifecycle dates, logos, and access scoping. Client visibility is governed by `user_client_access`, so managers can see all clients while regular users only see assigned ones. The backend supports client CRUD operations, logo upload, and secure logo serving through `/client-logo/{filename}`. The file also notes that missing client registry entries can be derived from reports and logos, making the client registry partially self enriching.

User Modules' Conclusion

With that being said, it is clear that SHELDT Dashboard is not just a UI with attached scripts, but a tightly orchestrated platform with clearly defined processing stages, fallback chains, persistence logic, and output registration.

Admin Dashboard

The Admin Panel is the managerial side of the SHELDT Dashboard. Unlike the user dashboard, which focuses on operational workflows such as report generation and scanning, the Admin Panel focuses on governance, visibility, and control. It gives managers access to analytics, system logs, user administration, and client management. Access is restricted through role-based authorization, so only users with the manager role can use these routes and interfaces.

At the access control level, the Admin Panel depends on the platform's authentication model. Sessions are stored in both the `in-memory_SESSIONS cache` and the `auth_sessions` table. The backend enforces manager only access using `require_manager_session`, and the documented manager capabilities include:

- User and system analytics
- PowerBI charts with company data
- System logs
- Adding and listing users
- Adding, removing, listing , and editing clients.

A simplified flow is:

Manager Login ->Token Validation ->Admin Dashboard Access ->Analytics / Logs / Users / Clients

The information displayed on the admin dashboard includes:

- KPI cards for activity, generated reports, and user participation
- time-series activity charts

- report-type breakdown charts
- top users and top actions
- embedded Power BI charts
- recent logs with filters

This makes the Admin Panel both an analytics page and an audit page.

Analytics and KPI Logic

The analytics section is designed to summarize how the platform is being used over time. The backend computes this data mainly from two tables:

- **activity_logs**
- **generated_reports**

The analytics endpoint groups activity by date and type, then returns aggregated values for use in dashboard cards and charts. In simple terms, the analytics layer answers questions such as:

- how many actions were performed
- how many reports were generated
- which users were most active
- which report types were used most often
- how activity changed over time

Typical KPI values include:

- total user activity
- total generated reports
- active user participation count
- report type split
- top actions by frequency

These values can be described using simple calculations:

Data	Formula
Total Activities	Sum of activity logs
Total Reports Generated	Sum of reports generated in total
User Activity Share	User Share % = (actions by user / total actions) × 100
Report Type Share	Type Share % = (reports of one type / total reports) × 100
Daily Count	count(records grouped by date)

This data is then displayed on the frontend in charts and tables. The documentation also notes a custom SVG line or area chart for activity vs. reports, and a donut-style breakdown for report types.

Logs and Audit Trail

The Admin Panel also functions as an audit interface through the logs system. Every important backend action is recorded in the activity_logs table. These records store who performed the action, what was done, when it happened, and additional details. This allows managers to review system behavior from an operational and compliance point of view.

The logs endpoint supports filtering, which is important because the system handles many different workflows. Supported filter dimensions include:

- mode buckets:
 - auth
 - generation
 - downloads
 - management
 - all
- username
- action
- role
- date
- configurable result limit

This means the Admin Panel can be used to answer questions like:

- who generated a report
- who downloaded a file
- when a user account was created
- when a client entry was updated
- which actions happened in a specific time window

A simple log flow is:

User action in system ->Backend writes activity_logs entry ->Admin panel fetches filtered logs ->Manager reviews audit trail

Power BI Integration

The Admin Panel includes support for embedded Power BI charts. On page load, the frontend calls **/auth/powerbi-charts**, and the backend returns chart configuration and embed details. The report files mention that the system normalizes and deduplicates embed URLs before presenting them, which prevents repeated or malformed embedded views.

The Power BI integration is configurable through environment values such as:

- POWERBI_MODE
- POWERBI_WORKSPACE_ID
- POWERBI_REPORT_ID
- POWERBI_TENANT_ID
- POWERBI_SHELT_PAGE_1_NAME
- POWERBI_SHELT_PAGE_2_NAME
- POWERBI_TICKETS_PAGE_1_NAME
- POWERBI_TICKETS_PAGE_2_NAME
- POWERBI_DEFAULT_GROUP

This allows the Admin Panel to go beyond internal SQLite analytics and display external BI dashboards when needed.

User Administration

The User Administration section allows managers to create and inspect user accounts. This is one of the most important administrative functions because it controls who can access the system and which clients they can see.

On the frontend, the user management page is manager only and supports:

- listing existing users
- creating new users
- assigning role
- assigning client access

The documentation states that the creation form includes:

- email
- password
- selected client access list

On the backend, **POST /auth/users** validates email uniqueness and password policy before account creation. New users default to the user role unless another allowed role is explicitly specified. Client assignments are stored in **user_client_access**, which creates a manyto many mapping between users and clients.

GET /auth/users returns role, active flag, and client access summary so the manager can review current assignments.

An important access rule is that if **client_ids** is omitted during user creation, the backend assigns all existing clients by default. Regular users, however, only see the clients they are assigned to. Managers see all clients.

Client Management from the Admin Side

Although Client Info also exists as its own functional module, from the Admin Panel perspective it is part of managerial control. Managers can create, update, and delete client records, while non-managers are limited to viewing only assigned clients.

Logo support is also connected to this area **through /upload-logo and /client-logo/{filename}**.

The client-management side stores and exposes:

- client registry data
- lifecycle dates
- active state
- logo metadata
- service and usage metrics

This gives managers both administrative control and visibility into which clients are active in the system.

Relationship Between Admin Panel and Database

The Admin Panel is heavily database-driven. The main tables it depends on are:

- users → account information
- auth_sessions → session tracking
- activity_logs → audit trail
- generated_reports → report counts and metadata
- clients → client registry
- user_client_access → user-to-client permissions

This means the Admin Panel is not just a visual layer. It is a control interface on top of the platform's most important operational and governance tables.

Security and Authorization Model in the Admin Panel

The Admin Panel is protected by the same authentication model used by the rest of the platform, but with stricter authorization requirements. Passwords are stored using PBKDF2-HMAC-SHA256 with per-user salt, tokens are bearer tokens with configurable TTL, and session state is mirrored between memory and the database. Manager only endpoints are guarded through **require_manager_session**, which prevents normal users from accessing analytics, logs, user administration, and client modification routes.

In reporting terms, the admin security model can be summarized as:

- authentication proves the user is valid
- role authorization proves the user is a manager
- client mapping controls what non-managers can see
- audit logs record what administrative actions were taken

Admin Panel Conclusion

The Admin Panel is the management layer of the SHELDT Dashboard. The frontend loads multiple sources in parallel to present KPIs, charts, logs, and account data. In practice, the Admin Panel allows managers to monitor usage, audit actions, manage accounts, and control client visibility, making it a central part of the platform's operational governance.

Conclusion

The SHELDT Dashboard represents a comprehensive cybersecurity operations platform that brings together reporting, intelligence gathering, reconnaissance, and administrative control into a single, unified system. Its main value lies in its ability to automate complex, multi-step workflows and transform them into structured, consistent, and usable outputs for both analysts and management, it streamlines tasks that would otherwise be manual and time consuming.

The Admin Panel adds an additional layer of value by enabling monitoring, auditing, and control over the entire platform. Through analytics, logs, and user management, it provides visibility into system usage and ensures that operations remain organized and secure.

Overall, the SHELDT Dashboard delivers a robust and scalable solution for cybersecurity operations. By combining automation, structured data processing, and centralized management, it enhances efficiency, improves accuracy, and provides meaningful insights that support both technical analysis and strategic decision making.

Appendix

SHELT

Welcome back

Sign In

Use your company credentials to continue.

Email

Password

Sign In

Login page

SHELT Workspace

Signed in as rhasbini@shelt.com

[Admin Dashboard](#)[Logout](#)

Weekly Report

Upload one REVA scan and generate the weekly package.

Monthly Report

Upload 4 required weekly files (plus optional week 5) and generate monthly output.

Dark Web Leaks

Prepare dark web leak reconnaissance workflows and planned evidence collection.

Proxy Chain (Kali Linux)

Configure and validate proxy-chain routing templates for Kali-based investigation sessions.

Domain Listing

Scan for lookalike, typosquat, and homoglyph domains targeting a brand with DNS and CT log checks.

Scraping

Run impersonation scraping for a target company and export XLSX findings.

Google Dorking

Generate a structured Google dork query pack for investigation and external reconnaissance.

TTI Feed

Generate technology threat-intelligence HTML outputs and download one zipped TTI pack.

Previous Reports

Browse and download generated weekly, monthly, scraping, dorking, TTI, domain listing, and dark-web outputs.

User dashboard-1

Proxy Chain (Kali Linux)

Configure and validate proxy-chain routing templates for Kali-based investigation sessions.

Domain Listing

Scan for lookalike, typosquat, and homoglyph domains targeting a brand with DNS and CT log checks.

Scraping

Run impersonation scraping for a target company and export XLSX findings.

Google Dorking

Generate a structured Google dork query pack for investigation and external reconnaissance.

TTI Feed

Generate technology threat-intelligence HTML outputs and download one zipped TTI pack.

Previous Reports

Browse and download generated weekly, monthly, scraping, dorking, TTI, domain listing, and dark-web outputs.

Client Info

Track lifecycle dates, service volumes, and update each client logo in one place.

User dashboard-2

Weekly Report

Upload one REVA scan and optionally a tickets file to generate one weekly report package.

REVA Scan (.xlsx)

Choose File No file chosen

Tickets File (Optional)

Choose File No file chosen

If omitted or invalid, generation still runs.

Customer Name (Required)

Generate Weekly Report

Generated Files (Current Run)

Only files from this run are shown here.

No files generated in this run yet.

Weekly Report Generation Screen

Clipboard Font Alignment Number Styles Cells Editing Add-ins

om — CTI Report

1	2	A	B	C	D	E	F	G	H	I	J	K
1	Full Name — CTI Report					SHELT						
2	Executive Summary March 25, 2026											
3												
4												
5	▼ Tiles summarize findings by risk and status											
6	4575 TOTAL		0 CRITICAL	2574 HIGH	516 MEDIUM	1301 LOW	184 INFO	0 SOLVED	1 ONGOING	3578 PENDING		
7												
8	Category	Sub-Category	Total	Solved	Ongoing	Pending	Latest	Risk Rating	Solved %	Status	Risk Description	
9	ANTI-PHISHING PROTECTION											
10	Anti-Phishing Protection	Active Suspicious Domains	6	0	0	0	6	High	0%	Active	Similar domains were found to be impersonating a customer and are luring customers to provide their confidential/financial data which may lead to reputational damage.	
11	Anti-Phishing Protection	Cousin Domains	10	0	0	0	0	Low	0%	Active		

Executive Dashboard

Latest Detections

Feedback Required

Tickets

Ongoing

Monitored List

Open Cases

Closed Cases

Findings Dashboard

Weekly report sample output snippet

Back to User Dashboard

Generate Report

Week 1

Choose File No file chosen

Week 2

Choose File No file chosen

Week 3

Choose File No file chosen

Week 4

Choose File No file chosen

Week 5 (Optional)

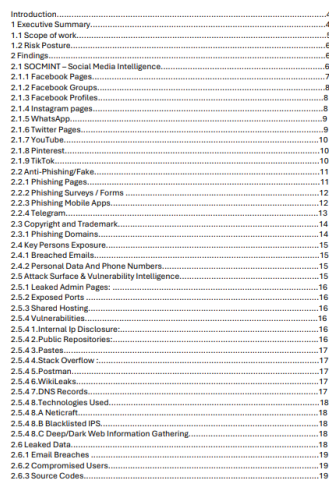
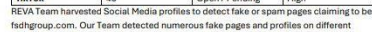
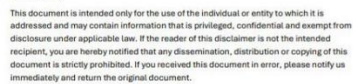
Choose File No file chosen

Generate Report

Generated Reports

No reports generated in this run yet.

Monthly Report Generation Screen



Monthly report sample output snippets

Back to User Dashboard

Dark Web Leak Intelligence
Detect leaked credentials, brand mentions, and impersonation domains

Company Name
xyz bank

Domain (optional)
bank.com

Keywords (comma separated)
leaks, admin, cards

☒ Dark Web Sources (.onion via Tor)
☒ Surface Web Sources
☒ Detect Impersonation

Tor Network
Connected

Exit IP: null — .onion sites will be searched

Run Scan

Scan History

N	COMPANY	DOMAIN	STATUS	CREATED	ACTIONS
---	---------	--------	--------	---------	---------

Dark web leaks screen snippet

Calibri

A1:F1

Dark web leaks output summary screen\

A1					
	A	B	C	D	E
1	#	Search Engine / Source	Query	Results Found	Executed At
2	1	DuckDuckGo (Paste Sites)	site:pastebin.com "gt bank"	8	2026-04-13 21:31:46
3	2	DuckDuckGo (Paste Sites)	site:paste.ee "gt bank"	8	2026-04-13 21:31:48
4	3	DuckDuckGo (Paste Sites)	site:pastebin.com "admin"	8	2026-04-13 21:31:51
5	4	DuckDuckGo (Paste Sites)	site:paste.ee "admin"	8	2026-04-13 21:31:53
6	5	DuckDuckGo (Paste Sites)	site:pastebin.com "internal"	8	2026-04-13 21:31:56
7	6	DuckDuckGo (Paste Sites)	site:paste.ee "internal"	8	2026-04-13 21:31:59
8	7	DuckDuckGo (Credentials)	gt bank data breach OR leaked OR dump	8	2026-04-13 21:32:02
9	8	DuckDuckGo (Dark Web)	gt bank darknet OR dark web OR hacked OR breach	8	2026-04-13 21:32:04
10	9	Abmia (Dark Web Index)	gt bank	20	2026-04-13 21:32:18
11	10	Abmia (Dark Web Index)	gt bank leak OR breach OR hack OR credentials	20	2026-04-13 21:32:21
12	11	Tor .onion Crawl	Deep crawled 10 .onion URLs for 'gt bank'	0	2026-04-13 21:33:00
13	12	Tor .onion Paste Sites	Checked 3 .onion paste sites for 'gt bank'	0	2026-04-13 21:33:07
14	13	Tor .onion Forums	Checked 2 .onion forums for 'gt bank'	0	2026-04-13 21:33:16
15					
16					
17					
18					
19					
20					
21					
22					
23					
Executive Summary Leaked Credentials Dark Web Mentions Impersonation Domains Search Queries Used					

Dark Web leaks output, queries sheet

#	Source	Type	Content / Snippet	URL
21	20ahmia_onion	Dark Web (.onion)	Chase Bank logs with cashapp paypal The Chase Bank Marketplace. Vouched	http://q3etpukg...nbpopuxedid.onion/product/365
22	21ahmia_onion	Dark Web (.onion)	Buy Sim... The Simple Bank Marketplace. Vouched	http://q3etpukg...nbpopuxedid.onion/product/365
23	22ahmia_onion	Dark Web (.onion)	Buy Sim... The Simple Bank Marketplace. Vouched	http://q3etpukg...nbpopuxedid.onion/product/365
24	23ahmia_onion	Dark Web (.onion)	Buy Sim... The Simple Bank Marketplace. Vouched	http://q3etpukg...nbpopuxedid.onion/product/365
25	24ahmia_onion	Dark Web (.onion)	Buy Sim... The Simple Bank Marketplace. Vouched	http://q3etpukg...nbpopuxedid.onion/product/365
26	25ahmia_onion	Dark Web (.onion)	Buy Sim... The Simple Bank Marketplace. Vouched	http://q3etpukg...nbpopuxedid.onion/product/365
27	26ahmia_onion	Dark Web (.onion)	Buy Sim... The Simple Bank Marketplace. Vouched	http://q3etpukg...nbpopuxedid.onion/product/365
28	27ahmia_onion	Dark Web (.onion)	Buy Sim... The Simple Bank Marketplace. Vouched	http://q3etpukg...nbpopuxedid.onion/product/365
29	28ahmia_onion	Dark Web (.onion)	Buy Sim... The Simple Bank Marketplace. Vouched	http://q3etpukg...nbpopuxedid.onion/product/365
30	29ahmia_onion	Dark Web (.onion)	Buy Sim... The Simple Bank Marketplace. Vouched	http://q3etpukg...nbpopuxedid.onion/product/365

Dark web leaks snippet

Back to User Dashboard

Domain Listing

Discover lookalike and typosquat domains

Enter a target company name to scan for registered lookalike domains, typosquats, and homoglyph variations. Results are enriched with DNS resolution, HTTP liveness checks, and Certificate Transparency data.

Target Company Name

Known Official Domains (optional)

Press Enter, comma, or space after each domain. These will be tagged as official in results.

Start Domain Scan

Scan Summary

Checked 158 candidate domains for **zenithbank**

HIGH RISK
2

MEDIUM
4

LOW RISK
0

OFFICIAL
2

NOT REGISTERED
150

Export to Excel

Domain listing screen

Discovered Domains (158 shown)							
DOMAIN	SOURCE	IP ADDRESS	HTTP	CERT	LIVE	RISK	
zenithbank.com	Official	45.60.50.43	200	—	●	Official	
zenithbank.org	Official	—	—	—	●	Official	
email.zenithbank.com	CT Log	41.203.113.128	—	✓	●	High Risk	
realtime.zenithbank.com	CT Log	45.60.46.43	503	✓	●	High Risk	
zenithbank.co.za	Variation	77.79.239.194	200	—	●	Medium	
zenithbank.net	Variation	15.197.148.33	405	—	●	Medium	
zenithbank.online	Variation	172.67.193.36	525	—	●	Medium	
zenthbank.com	Variation	52.20.84.62	403	—	●	Medium	
z3nithbank.app	Variation	—	—	—	●	Not Registered	
z3nithbank.co	Variation	—	—	—	●	Not Registered	

Domain listing generation output snippet

Google Dorking

Generate a ready-to-use investigation query pack for one target company. You can optionally scope queries to a specific domain.

Target Company Name

gt bank

Domain Scope (Optional)

Leave empty to generate broader brand and exposure queries.

Generate Dork Pack

Dorking pack generated successfully with 21 queries, 20 result links, and 15 screenshots. Collected 20 live result links and 15 screenshot evidence items. Engine distribution: Brave=5, Mojeek=7, Google-API=8. See the Query_Results sheet for full details.

Output Files (Current Run)

Excel output includes all generated queries, all collected result links/domains, and actual embedded screenshots in **Query_Results**.

gt_bank_google_dorking_April_2026.xlsx

Download

Collection Summary

TOTAL QUERIES

21

COLLECTED LINKS

20

EMBEDDED

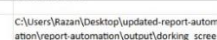
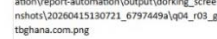
SCR
15

Previews were intentionally removed. Open the exported Excel file and review the full **Query_Results** sheet for complete evidence.

Google Dorking automation Screen

	A	B	C	D	E	F	G	H
1	Category	Purpose	Google Dork Query	Search Engine	Search URL	Result Rank	Result Title	Result URL
2	Brand Abuse	Brand + login/verification lures and phishing indicators	"gt bank" "login" OR "verify" OR "security update" OR "gt bank" "login" OR "verify" OR "security update" OR "reset password" OR "account suspended"	brave	https://www.google.com/search?q=%22gt+bank%22+%28%22login%22+OR+%22verify%22+OR+%22security+update%22+OR+%22reset+password%22+OR+%22account+suspended%22%29+-site%3Alinkedin.com+-site%3Afacebook.com+-site%3Atwitter.com+-site%3Ainstagram.com+-site%3Atiktok.com+-site%3Ayoutube.com+-site%3Areddit.com+-site%3APinterest.com+-site%3Ame	1	GTBank Login: A Complete Guide to Accessing Your Security Information GTBank UK	https://dsnews.co.uk/gtbank/login/
3	Brand Abuse	Brand + login/verification lures and phishing indicators	"gt bank" "login" OR "verify" OR "security update" OR "reset password" OR "account suspended"	brave	https://www.google.com/search?q=%22gt+bank%22+%28%22login%22+OR+%22verify%22+OR+%22security+update%22+OR+%22reset+password%22+OR+%22account+suspended%22%29+-site%3Alinkedin.com+-site%3Afacebook.com+-site%3Atwitter.com+-site%3Ainstagram.com+-site%3Atiktok.com+-site%3Ayoutube.com+-site%3Areddit.com+-site%3APinterest.com+-site%3Ame	2	Security Information GTBank Ghana	https://www.gtbankuk.com/security-c
4	Brand Abuse	Brand + login/verification lures and phishing indicators	"gt bank" "login" OR "verify" OR "security update" OR "reset password" OR "account suspended"	brave	https://www.google.com/search?q=%22gt+bank%22+%28%22login%22+OR+%22verify%22+OR+%22security+update%22+OR+%22reset+password%22+OR+%22account+suspended%22%29+-site%3Alinkedin.com+-site%3Afacebook.com+-site%3Atwitter.com+-site%3Ainstagram.com+-site%3Atiktok.com+-site%3Ayoutube.com+-site%3Areddit.com+-site%3APinterest.com+-site%3Ame	3	Security Information GTBank Gambia	https://www.gtbankgambia.com/help-cent
5	Brand Abuse	Brand + login/verification lures and phishing indicators	"gt bank" "login" OR "verify" OR "security update" OR "reset password" OR "account suspended"	brave	https://www.google.com/search?q=%22gt+bank%22+%28%22login%22+OR+%22verify%22+OR+%22security+update%22+OR+%22reset+password%22+OR+%22account+suspended%22%29+-site%3Alinkedin.com+-site%3Afacebook.com+-site%3Atwitter.com+-site%3Ainstagram.com+-site%3Atiktok.com+-site%3Ayoutube.com+-site%3Areddit.com+-site%3APinterest.com+-site%3Ame	1	GTBank Customer Service Number +234 448	https://www.gtbankgambia.com/secu
6	Brand Abuse	Brand + login/verification lures and phishing indicators	"gt bank" "login" OR "verify" OR "security update" OR "reset password" OR "account suspended"	brave	https://www.google.com/search?q=%22gt+bank%22+%28%22login%22+OR+%22verify%22+OR+%22security+update%22+OR+%22reset+password%22+OR+%22account+suspended%22%29+-site%3Alinkedin.com+-site%3Afacebook.com+-site%3Atwitter.com+-site%3Ainstagram.com+-site%3Atiktok.com+-site%3Ayoutube.com+-site%3Areddit.com+-site%3APinterest.com+-site%3Ame	2	GTBank Customer Service Number +234 448	https://gtbank.pisscessconsumer.com/cu
7	Data Exposure	Target-themed leaked files, datasets, Lookalike and typosquat domains registered to impersonate the target	"gt bank" "flytpepxls OR flytpepxors OR flytpepxor OR "gt bank" "whioir" OR "domain registration" OR "typosquat" OR "lookalike" OR "impersonation" OR "fake website" -site:linkedin.com -site:facebook.com -site:twitter.com -site:x.com -site:instagram.com -site:tiktok.com -site:youtube.com -site:reddit.com -site:pinterest.com -site:me	google-api	https://www.google.com/search?q=%22gt+bank%22+%28%22login%22+OR+%22verify%22+OR+%22security+update%22+OR+%22reset+password%22+OR+%22account+suspended%22%29+-site%3Alinkedin.com+-site%3Afacebook.com+-site%3Atwitter.com+-site%3Ainstagram.com+-site%3Atiktok.com+-site%3Ayoutube.com+-site%3Areddit.com+-site%3APinterest.com+-site%3Ame	1	[PDF] Case Study: GT Bank ProcessMaker	https://www.processmaker.com/wp-c
8	Impersonation Domain	Fake or trojanized mobile apps mimicking the target brand	"gt bank" "fake app" OR "malicious app" OR "trojan" OR "apk download" OR "sideload" (android ios oss OR mobile) -site:linkedin.com -site:facebook.com -site:twitter.com	google-api	https://www.google.com/search?q=%22gt+bank%22+%28%22login%22+OR+%22verify%22+OR+%22security+update%22+OR+%22reset+password%22+OR+%22account+suspended%22%29+-site%3Alinkedin.com+-site%3Afacebook.com+-site%3Atwitter.com+-site%3Ainstagram.com+-site%3Atiktok.com+-site%3Ayoutube.com+-site%3Areddit.com+-site%3APinterest.com+-site%3Ame	1	How To Use Gtbank Mobile App - ScholarShipair	https://technet24.com/2024/08/15/e

Google Dorking sample output snippet-1

	P	G	H	I	J	K	L	M	N
	Result Rank	Result Title	Result URL	Result Domain	Screenshot Path	Screenshot			
2	1	GTBank Login: A Complete Guide to Accessing Your Security Information GTBank UK	https://dsnews.co.uk/gtbank-login/	dsnews.co.uk					
3	2	Security Information GTBank Ghana	https://www.gtbank.com/security-centre/security-inform	gtbank.com	C:\Users\Razan\Desktop\updated-report-automation\report-automation\output\dorking_screenshots\20260415130721_6797449a\q04_r02_gtbankum.com.png				
4	3	Security Information GTBank Gambia	https://www.gtbank.com/help-centre/security-informatic	gtbank.com	C:\Users\Razan\Desktop\updated-report-automation\report-automation\output\dorking_screenshots\20260415130721_6797449a\q04_r03_gtbankum.com.png				
5	1	Security Information GTBank Gambia	https://www.gtbankgambia.com/security-centre/security-in	gtbankgambia.com					
6	2	GTBank Customer Service Phone Number +2 341 448	https://gtbank.pissedconsumer.com/customer-service.html	gtbank.pissedconsumer.com					
7	1	POF Case Study: GT Bank ProcessMaker	https://www.processmaker.com/wp-content/uploads/2019/processmaker	processmaker.com					
8	1	GTBank: The scary implications of domain hijacking - Technext	https://technext24.com/2024/08/15/explaining-domain-hij	technext24.com	C:\Users\Razan\Desktop\updated-report-automation\report-automation\output\dorking_screenshots\20260415130721_6797449a\q14_r01_technext24.com.png				
9	1	How To Use Gtbank Mobile App - ScholarshipAid	https://technext24.com/2024/08/15/explaining-domain-hij	technext24.com	C:\Users\Razan\Desktop\updated-report-automation\report-automation\output\dorking_screenshots\20260415130721_6797449a\q15_r01_s				

Google Dorking sample output snippet-2

[Back to User Dashboard](#)

Scraping

Run impersonation scraping for a target company and export an XLSX file for weekly workflows. The scraping runs in the background — you can navigate away and come back.

Target Company Name

razan hasbini

Generate Scraping Report

Scraping report generated successfully.

Start New Scraping

Generated Files (Current Run)

Only files from this run are shown here.

razan_hasbini_scraping_April_2026.xlsx

Download

Scraping Screen

	A	B	C	D	E	F	G
	TIKTOK	razanhasbini	https://www.tiktok.com/@razanhasbini				
4							
	INSTAGRAM	razan_hasbini	https://www.instagram.com/razan_hasbini/				
5							
	INSTAGRAM	razanhasbini	https://www.instagram.com/razanhasbini/				

Scraping output snippet

Proxy Settings

Choose how your scans connect to the internet



Auto-Proxy
Automatically scrapes free proxies from the internet, tests them, and uses a working one



Auto-proxy is enabled — click "Test & Find Working Proxy" to find one now, or it will find one automatically when you run a scan

Proxy Type

HTTP

Testing proxies...

Refresh Proxy Pool

Sources: TheSpeedX, monosans, ShiftyTR, clarkern, hookzof, roosterkid proxy lists on GitHub. These are updated every few hours with thousands of fresh free proxies.



Add Your Trusted Proxy
Have your own proxy? Add it here — it will be used directly with no rotation

Type

SOCKS5

Host / IP Address

000.0.0.0

Port

1080

Username (if required)

optional

Password (if required)

optional

Proxy settings screen snippet-1

optional

optional

Test Connection

Save Proxy

Your Saved Proxies

These are your manually saved proxies. Select any of them from the "Proxy" dropdown on module pages (Dorking, Scraping, Dark Web, etc.)

HTTP : //35.209.198.222:80

Dead

Last test failed — this proxy may be down

Test

Edit

Delete

Advanced: Pool Management & Config Export

Import bulk proxy lists, validate proxies, and export ProxyChains configs

Select a proxy to manage its pool

— Choose —

Proxy Settings Screen-2

Back to User Dashboard

Previous Reports

Browse and download previously generated weekly, monthly, scraping, Google dorking, TTI, domain listing, and dark-web outputs.

Type

Month

Year

All

All

All

REPORT	TYPE	MONTH	YEAR	LAST MODIFIED	DOWNLOADS
ttdispatchttiApril2026314b6274	TTI	April	2026	4/15/2026, 1:24:13 PM	ZIP
gtbankgoogle_dorking_April_2026	Google Dorking	April	2026	4/15/2026, 1:13:46 PM	XLSX ZIP
razanhasbiniscraping_April_2026	Scraping	April	2026	4/15/2026, 12:55:54 PM	XLSX
sidf_April_2026	Monthly	April	2026	4/15/2026, 12:39:18 PM	PDF DOCX
DarkWeb_Report_gt_bank_8_091311	Dark Web	April	2026	4/15/2026, 12:13:11 PM	XLSX
DarkWeb_Report_gt_bank_8	Dark Web	April	2026	4/15/2026, 12:13:11 PM	XLSX

Previous Reports screen

Back to Admin Dashboard

Refresh

Client Info

Manage client lifecycle details, logo assets, and service generation totals. Counts update automatically from weekly, monthly, scraping, dorking, and TTI outputs.

Clients

29

Weekly

2

Monthly

11

Scraping + Dorking + TTI

46

Total Services

63

Selected Client Metrics

Select any client row below to see detailed counts and lifecycle info.

Weekly

0

Monthly

0

Scraping

0

Dorking

2

TTI

0

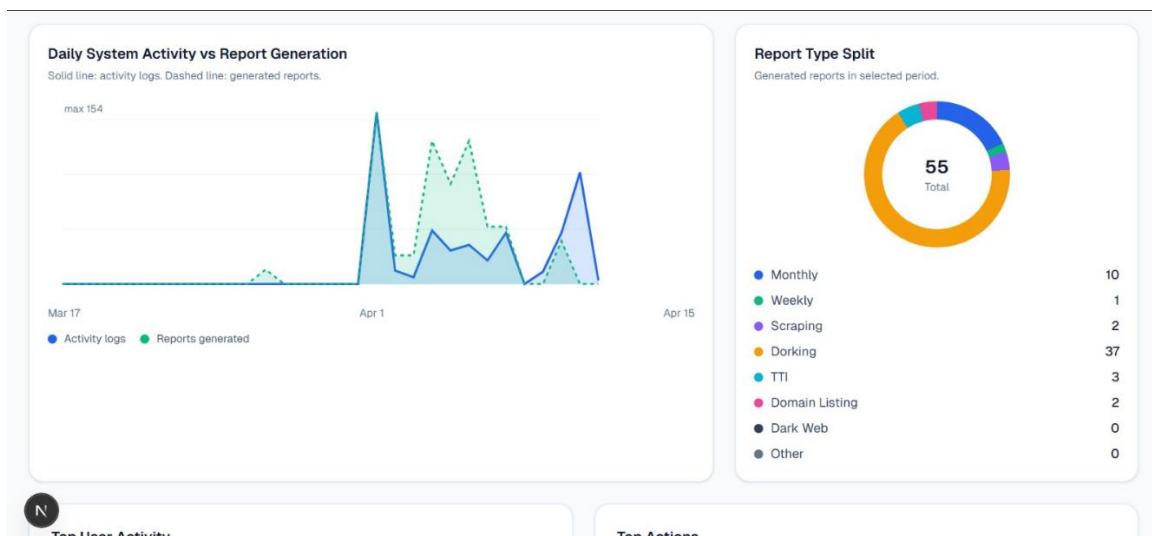
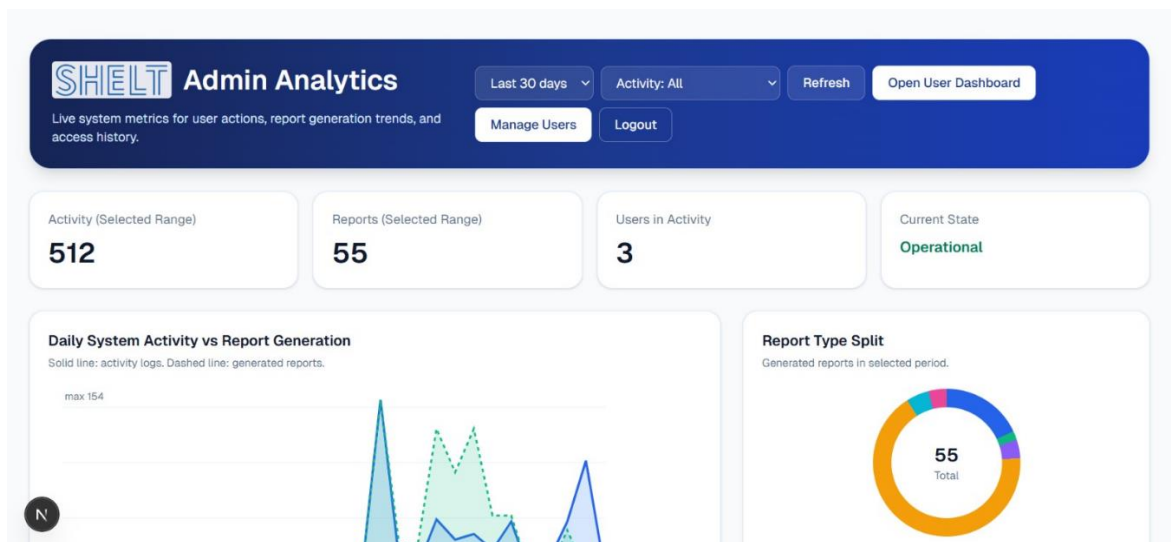
Total Services

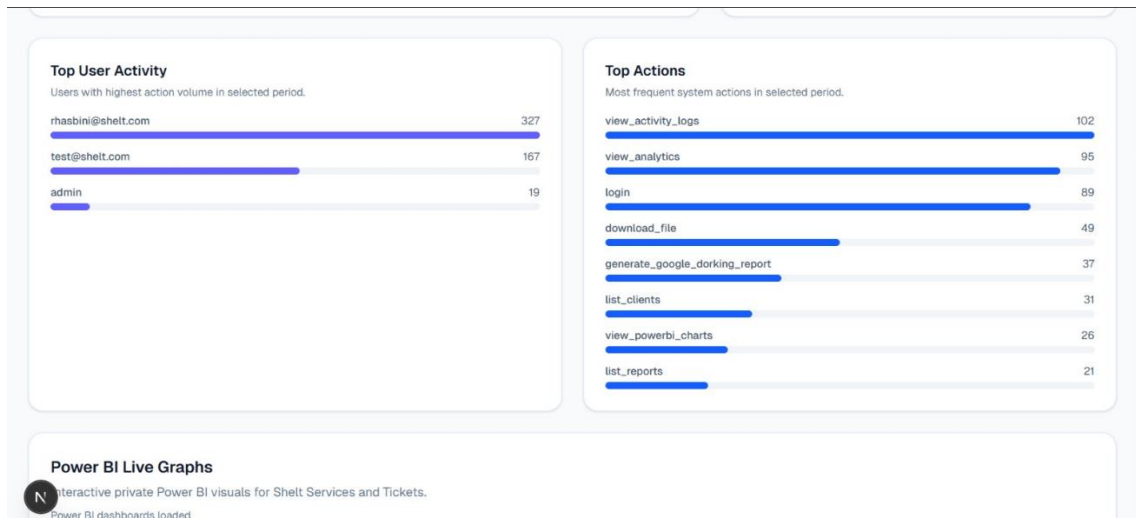
2

Firstbank
Started: 4/4/2026 | Left: -

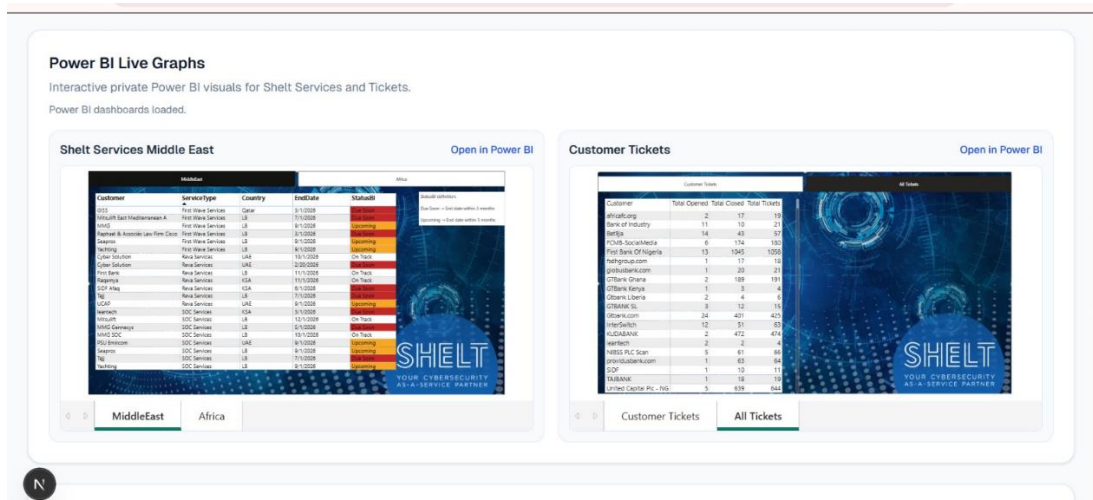
Client info screen -1

	Bet9Ja slug: bet9ja logo: bet9ja.jpg	ACTIVE	-	-	0	0	0	0	0	Edit Remove
	Boi slug: boi logo: none	ACTIVE	4/10/2026	-	0	0	0	1	1	Edit Remove
	Fcmb slug: fcmb logo: FCMB.png	ACTIVE	-	-	0	0	0	0	0	Edit Remove
	Firstbank slug: firstbank logo: firstbank.png	ACTIVE	4/4/2026	-	0	0	0	2	2	Edit Remove
	Globusbank slug: globusbank logo: globus bank.png	ACTIVE	-	-	0	0	0	0	0	Edit Remove
	Gtbank slug: gtbank logo: gtbank.png	ACTIVE	4/3/2026	-	1	0	2	17	22	Edit Remove





Admin Dashboard – 3



Admin Dashboard -4

Recent Activity Logs

Latest authenticated actions performed in the system. Mode: All

Open advanced log filters

User contains

Action contains

Role

Date from

Date to

Rows

All Roles

mm/dd/yyyy

mm/dd/yyyy

200

Apply filters

Reset

Time	User	Role	Action	Details
4/15/2026, 12:02:06 PM	rhasbini@shelt.com	Manager	view_powerbi_charts	enabled=True; groups=2; selected_group=shelt_services; charts=2
4/15/2026, 12:02:06 PM	rhasbini@shelt.com	Manager	view_analytics	days=30
4/15/2026, 12:02:06 PM	rhasbini@shelt.com	Manager	view_activity_logs	mode=all; limit=200; user=-; action=-; role=all; date_from=-; date_to=-; returned=200
4/14/2026, 9:46:54 PM	rhasbini@shelt.com	Manager	view_analytics	days=30
4/14/2026, 9:46:54 PM	rhasbini@shelt.com	Manager	view_activity_logs	mode=all; limit=200; user=-; action=-; role=all; date_from=-; date_to=-; returned=200
4/14/2026, 9:46:54 PM	rhasbini@shelt.com	Manager	view_analytics	days=30
4/14/2026, 9:46:54 PM	rhasbini@shelt.com	Manager	view_powerbi_charts	enabled=True; groups=2; selected_group=shelt_services; charts=2
4/14/2026, 9:46:54 PM	rhasbini@shelt.com	Manager	view_activity_logs	mode=all; limit=200; user=-; action=-; role=all; date_from=-; date_to=-; returned=200
4/14/2026, 9:46:54 PM	rhasbini@shelt.com	Manager	view_powerbi_charts	enabled=True; groups=2; selected_group=shelt_services; charts=2

Admin Dashboard-5

Manage Users

Create login accounts for coworkers. Signed in as rhasbini@shelt.com.

Email

Password

Client Access (Dropdown)

29 selected

▼ Select clients this user can access

☒

Abbeymortgagebank

☒

Accessbank

☒

Bankmed

☒

Bankofindustrybic

☒

Bet9Ja

☒

Boi☒☒

Existing Users

Accounts currently allowed to sign in.

admin

Active • Role: manager • Created 3/28/2026, 3:40:05 PM

Clients: No access assigned

rhasbini@shelt.com

Active • Role: manager • Created 3/28/2026, 4:04:20 PM

Clients: No access assigned

test@shelt.com

Active • Role: user • Created 4/3/2026, 10:59:29 AM

Clients: No access assigned

Admin Dashboard- Manage Users Screen